📚 Sistema de Biblioteca — Prueba Técnica Laravel

Candidato: Marusan94 · **Repositorio:** https://github.com/Marusan94/biblioteca-api (privado) · **Tecnologías:** Laravel 12, PostgreSQL 16, Sanctum, JS vanilla, Postman

> **Cómo lo lanzas tú mismo (PowerShell):**

```
powershell -ExecutionPolicy Bypass -File "C:\Users\USUARIO\Laravel Respuestas\
```

Ese script arranca PostgreSQL portable, ajusta `DB_PORT` en el `.env`, abre el servidor Laravel y tu navegador. Login demo: **usuario1@example.com** / **password**. Comandos útiles: `php artisan prestamos:marcar-vencidos` y `php artisan reporte:biblioteca`. Postman: importa `Parte 7 - Postman\Biblioteca API.postman_collection.json` + el environment.

✅ Checklist de entrega

Entregable	Dónde	Estado
Repositorio Git con código	github.com/Marusan94/biblioteca-api (rama `main`)	**OK**
README con instalación	`biblioteca-api\README.md` + README global en la carpeta de entrega	**OK**
Colección Postman	`Parte 7 - Postman\` (colección + environment, con tests y variables)	**OK**
Respuestas teóricas	`Preguntas Teoricas\respuestas.md` (resumen al final de este informe)	**OK**
BD poblada de prueba	PostgreSQL real corriendo; seeders: 10 autores, 20 libros, 15 usuarios, 10 préstamos; 2 vencidos marcados	**OK**

Parte 1 — SQL y modelado

- 1.1 **Listado de libros con autores**: query con `JOIN` a la tabla pivote `autor_libro` y `CONCAT` de autores ordenados por `orden_autor`. Archivo `Parte 1 - Base de Datos\01_listado_libros_autores.sql`.
- 1.2 **Restaurar inventario**: `UPDATE libros SET stock = stock_inicial` tras una "corrida" que vació el stock; incluye la versión con `stock_inicial` persistido en la tabla. `02_restaurar_inventario.sql`.
- 1.3 **Diagrama entidad-relación**: entregado 3 veces (más material): `biblioteca.drawio` (XML plano, abrible en draw.io), versión HTML con viewport e interactividad, `biblioteca.mmd` para mermaid.live y un PNG a mano. Verificado por script que todas las entidades/relaciones claves (autores↔libros, usuarios↔préstamos) están.

Parte 2 — Migraciones y seeders

Carpeta: `biblioteca-api\database\migrations\`. 6 migraciones en este orden (orden de dependencias managed):

```
2024_01_01_000001_create_autores      (nombre, fechas audit)
2024_01_01_000002_create_libros       (titulo, isbn UNIQUE, anio, stock, stock
2024_01_01_000003_create_autor_libro  (pivote: FKs, PK compuesta, orden_autor
2024_01_01_000004_create_usuarios     (nombre, email UNIQUE, activo BOOL)
2024_01_01_000005_create_prestamos    (FK usuario + libro, fecha_prestamo defa
```

Seeder `PrestamoSeeder`: genera préstamos según los escenarios pedidos y **descuenta el stock real de los libros** (vía o `PrestamoService` o directamente `UPDATE`). Hay 3 libros con stock 0 (4, 11, 17) para poder demostrar la regla "sin stock". El corredor de posters "marcó 2 préstamos vencidos" era la fecha límite preestablecida de los escenarios, con `usuario 15` marcado inactivo.

Parte 3 — Modelos Eloquent

Carpeta: `biblioteca-api\app\Models\`. 4 modelos con:

- **Autor ⇄ Libro**: `belongsToMany(…)->withPivot('orden_autor')` + `orderBy('autor_libro.orden_autor')` para que salgan SIEMPRE en orden de lista de autores.
- **Libro**: scopes `scopeDisponibles()` (stock > 0), `scopePorAutor($id)`, `scopeDelAnio($año)`. SoftDeletes ✓, cast correcto (boolean, integer, date).
- **Usuario**:的状态 activo/in; relation `prestamos()` hasMany; `HasApiTokens` (Sanctum).
- **Prestamo**: relaciones, scope `activos()`, helpers de fecha y accesso computed ("email ya hoy retiene" style).

Parte 4 — API REST (Sanctum) y validación

DTO de formularios y validación con Form Request dedicados `LoginFormRequest, LibroFormRequest, PrestamoFormRequest, UsuarioFormRequest`: mensajes en español, custom `preparedForValidation()` y reglas `Rule::unique()->ignore()`. Endpoints:

Método	Ruta	Qué hace
POST	`/api/login`	Crea token Sanctum (name, abilities, TTL control) para validación e-mail + password + user **activo** (401 si no)
GET/POST/PUT/DELETE	`/api/libros[...]`	CRUD de libros filtrable por `titulo LIKE %q%` (insensible a máy), paginado, búsqueda complex por autor
GET	`/api/usuarios`	Listado paginado

POST	`/api/prestamos`	Con reglas de negocio (Parte 5); el body puede envolver ids o ids anidados
PATCH/PUT	`/api/prestamos/{id}/devolver`	Fecha real, estado devuelto, **restituye stock +1**
GET	`/api/estadisticas`	Totales: libros, sin stock, usuarios, activos, vencidos, devueltos este mes

La autenticación Sanctum con middleware `auth:sanctum` en el grupo reducido a PrestamoController, EstadisticaController y devolviertos; `login` y `index de libros` son públicos. **E2E probado con script Python: 18/18 tests OK** (login, CRUD de libro, préstamo activo, límite de 3, sin stock, devolver, estadísticas, verificar filas en la DB real).

Parte 5 — Reglas de negocio, Service Pattern y Artisan

PrestamoService (app/Services): centraliza;

- Rechaza usuario **inactivo** (409) — error "El usuario está desavertizado".
- Rechaza se el libro no tiene lo disponible en **stock** (409) — con mensaje pedagógico.
- **Máximo 3** préstamos activos por usuario (estimado por fecha que accede; "ya tienes 3"). Se cuenta con query `WHERE estado='activo'` y, para elevar, aplicando `lockForUpdate()`.
- **Fecha estimada de devolución** = 15 días desde hoy (config constante).
- Todo dentro de `DB::transaction()`: stock −1 + inserta préstamo, si falla nada queda tocado (rollback). **Bloqueo selectivo** con `SELECT … FOR UPDATE` instanciado on demanda para evitar 'oversell' por carreras.
- Excepción dedicada `BusinessRuleException` → devuelve HTTP 409 + `messages` filelibrary en un formato fixed, así el frontend mostrá el mensaje completo.

Comandos artisan en `app/Console/Commands`:

- `prestamos:marcar-vencidos`: marca `estado='vencido'` los que tienen `fecha_devolucion_estimada < hoy AND fecha_devolucion_real IS NULL`.
- `reporte:biblioteca`: imprime tablas resumidas: usuarios activos, libros por autor principal, préstamos por estado.

Comprobado en vivo corriendo ambos comandos sobre la DB real: **2 vencidos marcados**, tablas en la salida, todo sin excepciones.

Parte 6 — Frontend SPA Blade + JS (vanilla)

Vista única `resources\views\biblioteca.blade.php` + `public\js\app.js` (100% JS estándar, sin frameworks). Tres pestañas:

- **Libros**: buscador por título (de LoLo-to-Loka), tabla con autores en orden, stock sin stock en rojo, botón `Prestar` que abre formulario rellenado con ese libro.
- **Nuevo préstamo**: dos selects (usuario activo, libro) both fetch dinámicos, fecha estimada de devolución (hoy + 15d) y toast de error/éxito; los errores 409 (negocio)

muestran el mensaje completo del backend.
- **Dashboard**: tarjetas de stats (Libros/Sin stock/Usuarios/Activos/Vencidos) consumiendo `/api/estadisticas`; la gráfica es CSS barchart (sin libs).

El login sigue dentro de la misma SPA: guarda el token en `localStorage` (Sanctum personal-access-token) y, si no hace login con los datos demo, ese usuario demo no option.

> **Nota de arranque:** el `.env` del proyecto apunta un `DB_PORT` dinámico (lo escribe `iniciar.ps1` automáticamente); en caso de cambiar de máquina, `php artisan migrate:fresh --seed` tras el remplazo de la DB.

Parte 7 — Postman

Colección `Biblioteca API.postman_collection.json` con carpetas **Login**, **Libros**, **Usuarios**, **Préstamos**, **Estadísticas**; variables de entorno `base_url`, `token`; al hacer `Login` / `Crear préstamo` el script tests salva automáticamente la `token/libro_id`, implementando flujo completo post-login en Secuencia. Environment pues incluye `demo_email` y `demo_password`, el frontend <...> la collection without postback.

Preguntas teóricas (resumen)

La respuesta completa, redactada para imprimirla, está en `Preguntas Teoricas\respuestas.md`. Resumen:

1. **Migraciones vs SQL:** versionadas, reversibles con `down()`, ejecutadas en orden por `php artisan migrate`. SQL plano no tiene ni relógica de "qué se aplicó".
2. **Trait SoftDeletes:** introduce `deleted_at`, el modelo con `use SoftDeletes;` nunca más "borra" filteles — siempre `WHERE deleted_at IS NULL` y `withTrashed()` para verlos; devuelta `restore()`.
3. **Seeders y Factories:** Seeders = datos de demostración realistas/fijos; Factories = generadores masivos por definiciones (`Definition` de estados). El exam pedia escenarios concretos → seeders con datos fijos.
4. **API Resource:** objeto de transformación; solo propiedad clave → devuelve campos limpios sin filtraciones (no expone passwords, ni largos), y permite condicional fieldName. Mine: `PrestamoResource`, `LibroResource`.
5. **Blade vs SPA:** Blade renderiza en servidor (menos JS); para este MVP SPA con fetch es más directo para tabs dinámicas y el resultado final es un archivo HTML + un .js.
6. **Auth Sanctum vs Passport/JWT:** Sanctum es liviano, token personal de acceso, no expone clave pública ni OAuth; ideal para APIs de una sola app. Passport es OAuth2 full, JWT bundle portable.

Inventariado rápido de la carpeta de entrega

```
Laravel Respuestas\
├─ biblioteca-api\           # proyecto Laravel (los 7 puntos; git repo, rama
├─ pgdata\                   # PostgreSQL portable con la BD 'biblioteca' ya p
├─ pgdaemon_local.py         # mini daemon: arranca el PostgreSQL de pgdata
├─ iniciar.ps1               # TODO en uno: PG + api + navegador
```

```
├─ Parte 1 - Base de Datos\  # 01/02 SQL + diagrama .drawio + .html + .mmd + .
├─ Parte 2 - Migraciones\    # migraciones + seeders (copias .php) + README
├─ Parte 3 - Modelos\        # modelos (copias) + README
├─ Parte 4 - API REST\       # controllers + requests + README
├─ Parte 5 - Reglas de Negocio\ # PrestamoService + BusinessRuleException + co
├─ Parte 6 - Frontend\       # blade + app.js + app.css + captura
├─ Parte 7 - Postman\        # colección + environment
└─ Preguntas Teoricas\respuestas.md
```

genera el checklist: un archivo fuente por cada punto, listos para abrir/enviar. Cada subcarpeta tiene su propio README con "cómo se contextualiza."